ПР 20.09 - T-SQL: управляющие конструкции, индексы и нормализация

- **Предметная область:** книжный интернет-магазин
- **СУБД:** Microsoft SQL Server 2025 (SSMS 22)
- **База данных:** BookStoreDB
- **Срок:** 20.09

Часть 1. Схема базы данных (3NF)

Связь многие-ко-многим книга - категории через BookCategories, позиции заказа в OrderItems.

Код

```
USE master;
GO
IF DB_ID(N'BookStoreDB') IS NOT NULL
BEGIN
    ALTER DATABASE BookStoreDB SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE BookStoreDB;
END
GO
CREATE DATABASE BookStoreDB;
GO
USE BookStoreDB;
GO
CREATE TABLE dbo.Authors (
    AuthorId  INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_Authors PRIMARY KEY,
    FullName  NVARCHAR(100) NOT NULL,
    Country   NVARCHAR(50)  NULL,
    BirthYear INT NULL CONSTRAINT CHK_Authors_BirthYear CHECK (BirthYear
BETWEEN 1500 AND 2020)
);

CREATE TABLE dbo.Categories (
    CategoryId INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_Categories PRIMARY
KEY,
    Name       NVARCHAR(50) NOT NULL CONSTRAINT UQ_Categories_Name UNIQUE
);

CREATE TABLE dbo.Books (
```

```
    BookId      INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_Books PRIMARY KEY,
    Title       NVARCHAR(200) NOT NULL,
    AuthorId    INT NOT NULL,
    Price       DECIMAL(10,2) NOT NULL CONSTRAINT CHK_Books_Price CHECK (Price
> 0),
    StockQty    INT NOT NULL CONSTRAINT DF_Books_StockQty DEFAULT 0
                CONSTRAINT CHK_Books_StockQty CHECK (StockQty >= 0),
    PublishYear INT NULL CONSTRAINT CHK_Books_PublishYear CHECK (PublishYear
BETWEEN 1450 AND 2100),
    CONSTRAINT FK_Books_Authors FOREIGN KEY (AuthorId) REFERENCES
dbo.Authors(AuthorId)
);

CREATE TABLE dbo.BookCategories (
    BookId     INT NOT NULL,
    CategoryId INT NOT NULL,
    CONSTRAINT PK_BookCategories PRIMARY KEY (BookId, CategoryId),
    CONSTRAINT FK_BookCategories_Books FOREIGN KEY (BookId) REFERENCES
dbo.Books(BookId),
    CONSTRAINT FK_BookCategories_Categories FOREIGN KEY (CategoryId)
REFERENCES dbo.Categories(CategoryId)
);

CREATE TABLE dbo.Customers (
    CustomerId INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_Customers PRIMARY KEY,
    FullName   NVARCHAR(100) NOT NULL,
    Email      NVARCHAR(100) NOT NULL CONSTRAINT UQ_Customers_Email UNIQUE,
    Phone      NVARCHAR(20)  NULL,
    City       NVARCHAR(50)  NOT NULL
);

CREATE TABLE dbo.Orders (
    OrderId    INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_Orders PRIMARY KEY
NONCLUSTERED,
    CustomerId INT NOT NULL,
    OrderDate  DATETIME2(0) NOT NULL CONSTRAINT DF_Orders_OrderDate DEFAULT
SYSDATETIME(),
    Status     NVARCHAR(20) NOT NULL CONSTRAINT DF_Orders_Status DEFAULT
N'Pending'
               CONSTRAINT CHK_Orders_Status CHECK (Status IN (N'Pending',
N'Shipped', N'Delivered', N'Cancelled')),
    CONSTRAINT FK_Orders_Customers FOREIGN KEY (CustomerId) REFERENCES
dbo.Customers(CustomerId)
);

CREATE TABLE dbo.OrderItems (
```

```
    OrderId   INT NOT NULL,
    BookId    INT NOT NULL,
    Quantity  INT NOT NULL CONSTRAINT CHK_OrderItems_Quantity CHECK (Quantity
> 0),
    UnitPrice DECIMAL(10,2) NOT NULL CONSTRAINT CHK_OrderItems_UnitPrice CHECK
(UnitPrice > 0),
    CONSTRAINT PK_OrderItems PRIMARY KEY (OrderId, BookId),
    CONSTRAINT FK_OrderItems_Orders FOREIGN KEY (OrderId) REFERENCES
dbo.Orders(OrderId),
    CONSTRAINT FK_OrderItems_Books FOREIGN KEY (BookId) REFERENCES
dbo.Books(BookId)
);
```

Часть 2. Заполнение данными

Код

```
INSERT dbo.Authors (FullName, Country, BirthYear) VALUES
(N'Лев Толстой', N'Россия', 1828),
(N'Фёдор Достоевский', N'Россия', 1821),
(N'Джордж Оруэлл', N'Великобритания', 1903),
(N'Агата Кристи', N'Великобритания', 1890),
(N'Стивен Кинг', N'США', 1947),
(N'Михаил Булгаков', N'Россия', 1891);

INSERT dbo.Categories (Name) VALUES
(N'Классика'),
(N'Роман'),
(N'Антиутопия'),
(N'Детектив'),
(N'Ужасы');

INSERT dbo.Books (Title, AuthorId, Price, StockQty, PublishYear) VALUES
(N'Война и мир', 1, 1200.00, 15, 1869),
(N'Анна Каренина', 1, 850.00, 7, 1877),
(N'Преступление и наказание', 2, 650.00, 0, 1866),
(N'Братья Карамазовы', 2, 990.00, 5, 1880),
(N'1984', 3, 550.00, 25, 1949),
(N'Скотный двор', 3, 390.00, 3, 1945),
(N'Убийство в Восточном экспрессе', 4, 990.00, 12, 1934),
(N'Оно', 5, 990.00, 0, 1986),
(N'Мастер и Маргарита', 6, 790.00, 9, 1967);

INSERT dbo.BookCategories (BookId, CategoryId) VALUES
(1, 1), (1, 2),
```

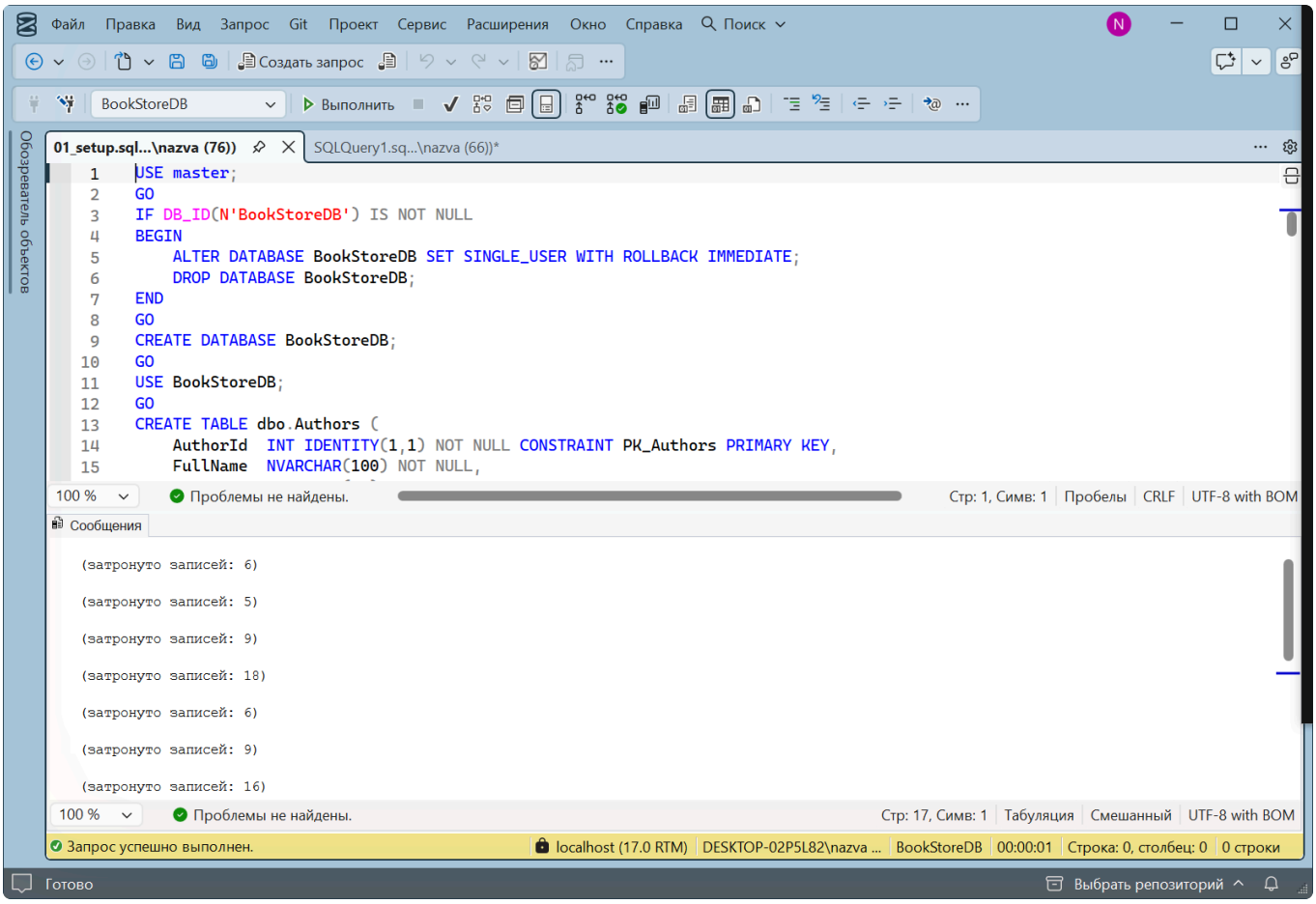
```
(2, 1), (2, 2),
(3, 1), (3, 2), (3, 4),
(4, 1), (4, 2),
(5, 2), (5, 3),
(6, 3),
(7, 4), (7, 2),
(8, 5), (8, 2),
(9, 1), (9, 2);

INSERT dbo.Customers (FullName, Email, Phone, City) VALUES
(N'Иван Петров', N'petrov@mail.ru', N'+7-901-111-11-11', N'Москва'),
(N'Мария Сидорова', N'sidorova@mail.ru', N'+7-902-222-22-22', N'Санкт-
Петербург'),
(N'Алексей Кузнецов', N'kuznetsov@mail.ru', NULL, N'Москва'),
(N'Ольга Смирнова', N'smirnova@mail.ru', N'+7-904-444-44-44', N'Казань'),
(N'Дмитрий Орлов', N'orlov@mail.ru', N'+7-905-555-55-55', N'Новосибирск'),
(N'Екатерина Волкова', N'volkova@mail.ru', NULL, N'Москва');

INSERT dbo.Orders (CustomerId, OrderDate, Status) VALUES
(1, '2026-09-01 10:15', N'Delivered'),
(2, '2026-09-02 12:40', N'Delivered'),
(3, '2026-09-05 09:05', N'Shipped'),
(1, '2026-09-08 18:30', N'Pending'),
(4, '2026-09-10 14:20', N'Pending'),
(5, '2026-09-12 11:00', N'Cancelled'),
(6, '2026-09-15 16:45', N'Pending'),
(2, '2026-09-18 20:10', N'Shipped'),
(3, '2026-09-20 13:35', N'Pending');

INSERT dbo.OrderItems (OrderId, BookId, Quantity, UnitPrice) VALUES
(1, 1, 1, 1200.00), (1, 5, 2, 550.00),
(2, 3, 1, 650.00), (2, 6, 1, 390.00), (2, 8, 1, 990.00),
(3, 7, 3, 990.00),
(4, 2, 1, 850.00), (4, 4, 2, 990.00),
(5, 5, 1, 550.00),
(6, 6, 4, 390.00),
(7, 1, 1, 1200.00), (7, 3, 1, 650.00), (7, 5, 1, 550.00),
(8, 7, 1, 990.00), (8, 8, 2, 990.00),
(9, 4, 1, 990.00);
```

Скриншот 1. Скрипт создания и заполнения выполнен без ошибок

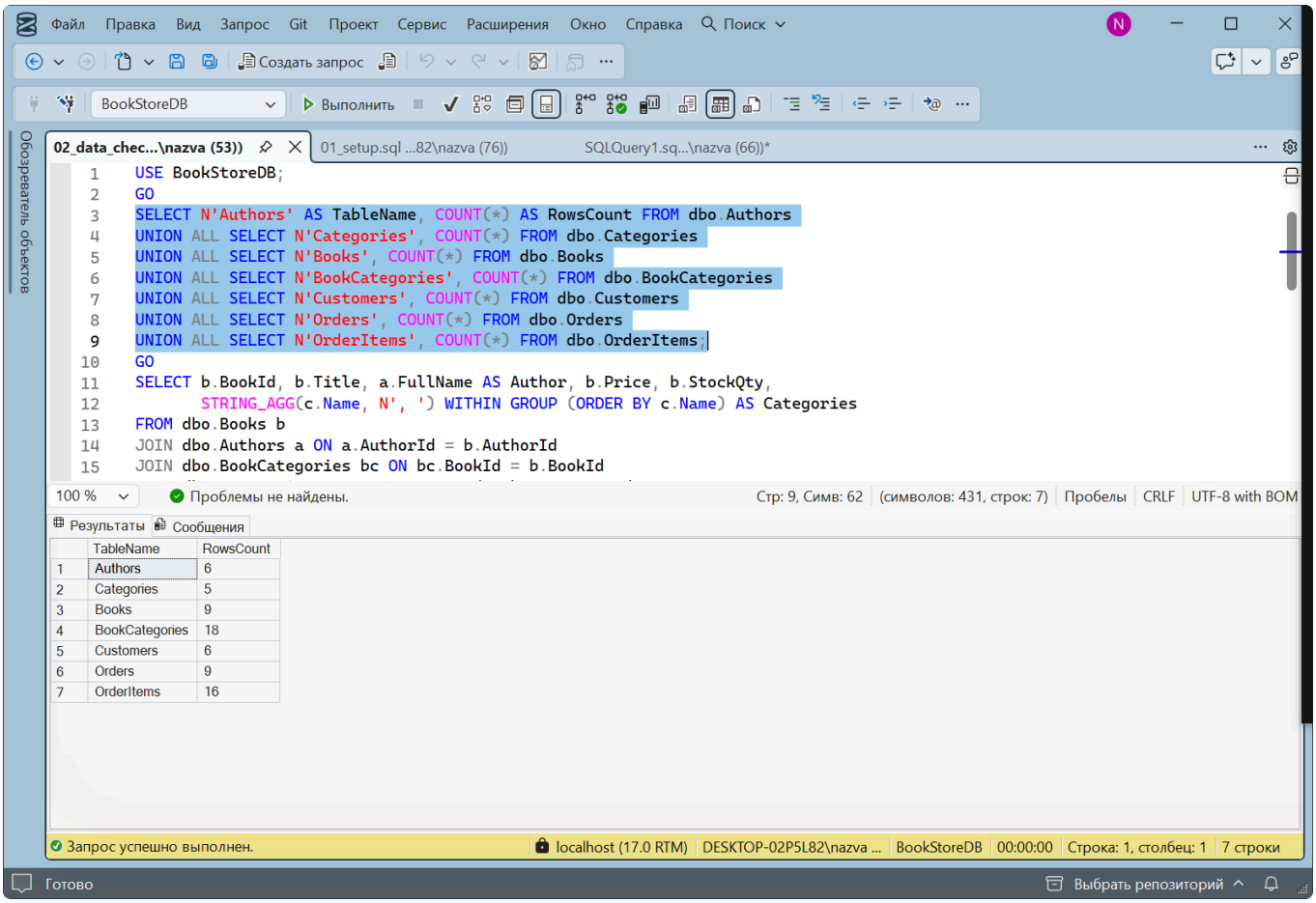


Тест

```
SELECT N'Authors' AS TableName, COUNT(*) AS RowsCount FROM dbo.Authors
UNION ALL SELECT N'Categories', COUNT(*) FROM dbo.Categories
UNION ALL SELECT N'Books', COUNT(*) FROM dbo.Books
UNION ALL SELECT N'BookCategories', COUNT(*) FROM dbo.BookCategories
UNION ALL SELECT N'Customers', COUNT(*) FROM dbo.Customers
UNION ALL SELECT N'Orders', COUNT(*) FROM dbo.Orders
UNION ALL SELECT N'OrderItems', COUNT(*) FROM dbo.OrderItems;
```

Результат: 9 книг, 6 авторов, 5 категорий, 6 клиентов, 9 заказов, 16 позиций.

Скриншот 2. Количество строк в таблицах



Часть 3. Практические задания

Задание 1. Индексы

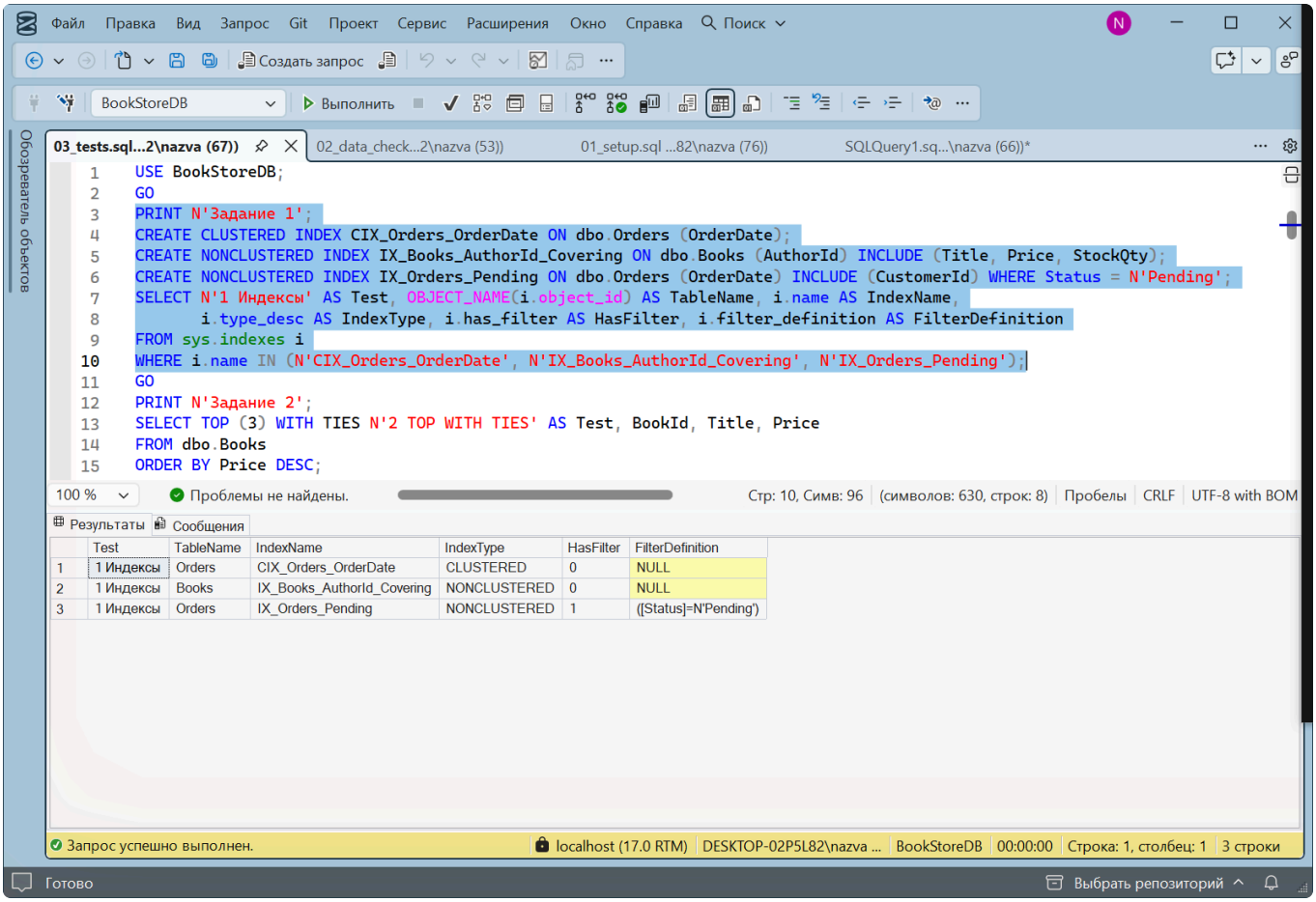
Код

```
PRINT N'Задание 1';
CREATE CLUSTERED INDEX CIX_Orders_OrderDate ON dbo.Orders (OrderDate);
CREATE NONCLUSTERED INDEX IX_Books_AuthorId_Covering ON dbo.Books (AuthorId)
INCLUDE (Title, Price, StockQty);
CREATE NONCLUSTERED INDEX IX_Orders_Pending ON dbo.Orders (OrderDate) INCLUDE
(CustomerId) WHERE Status = N'Pending';
SELECT N'1 Индексы' AS Test, OBJECT_NAME(i.object_id) AS TableName, i.name AS
IndexName,
       i.type_desc AS IndexType, i.has_filter AS HasFilter,
i.filter_definition AS FilterDefinition
FROM sys.indexes i
WHERE i.name IN (N'CIX_Orders_OrderDate', N'IX_Books_AuthorId_Covering',
N'IX_Orders_Pending');
```

Результат: созданы CIX_Orders_OrderDate (CLUSTERED), IX_Books_AuthorId_Covering (NONCLUSTERED), IX_Orders_Pending (NONCLUSTERED, фильтр ([Status]=N'Pending')).

- **CIX_Orders_OrderDate** - строки заказов физически упорядочены по дате, поэтому запросы за период (WHERE OrderDate BETWEEN ...) и сортировка по дате читают один непрерывный диапазон без сортировки.
- **IX_Books_AuthorId_Covering** - поиск книг автора идёт через Index Seek, а Title, Price, StockQty лежат в листьях индекса (INCLUDE), поэтому не нужен Key Lookup в таблицу.
- **IX_Orders_Pending** - индекс содержит только заказы со статусом Pending, он маленький и быстро отвечает на частый запрос «необработанные заказы», а при изменении других заказов не обновляется.

Скриншот 3. Созданные индексы из sys.indexes



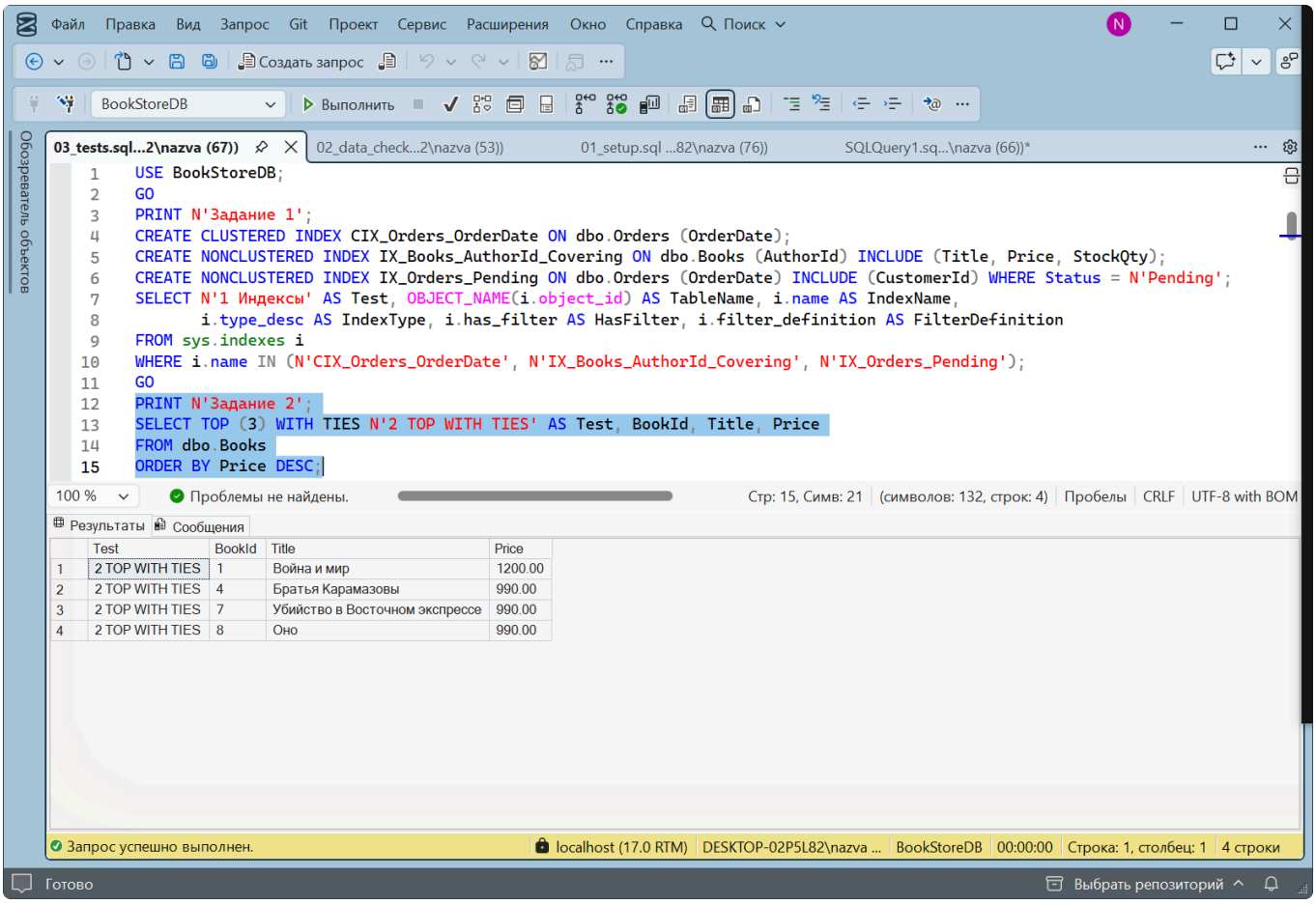
Задание 2. TOP с WITH TIES

Код

```
PRINT N'Задание 2';
SELECT TOP (3) WITH TIES N'2 TOP WITH TIES' AS Test, BookId, Title, Price
FROM dbo.Books
ORDER BY Price DESC;
```

Результат: 4 строки: «Война и мир» 1200.00 и три книги по 990.00 («Братья Карамазовы», «Убийство в Восточном экспрессе», «Оно») - на границе третьего места одинаковая цена, поэтому попали все.

Скриншот 4. Топ-3 самых дорогих книг с учётом равных цен



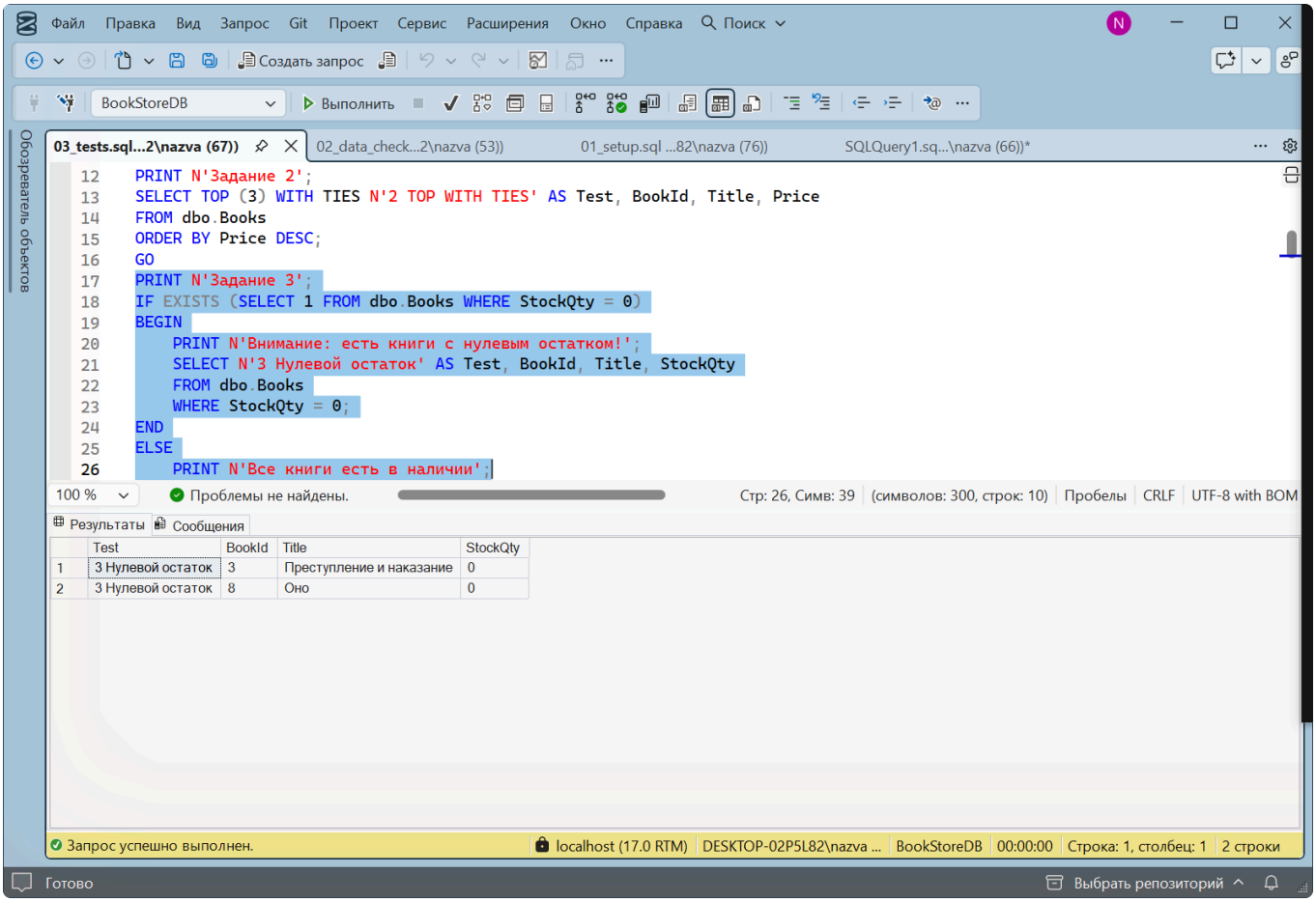
Задание 3. IF...ELSE + EXISTS

Код

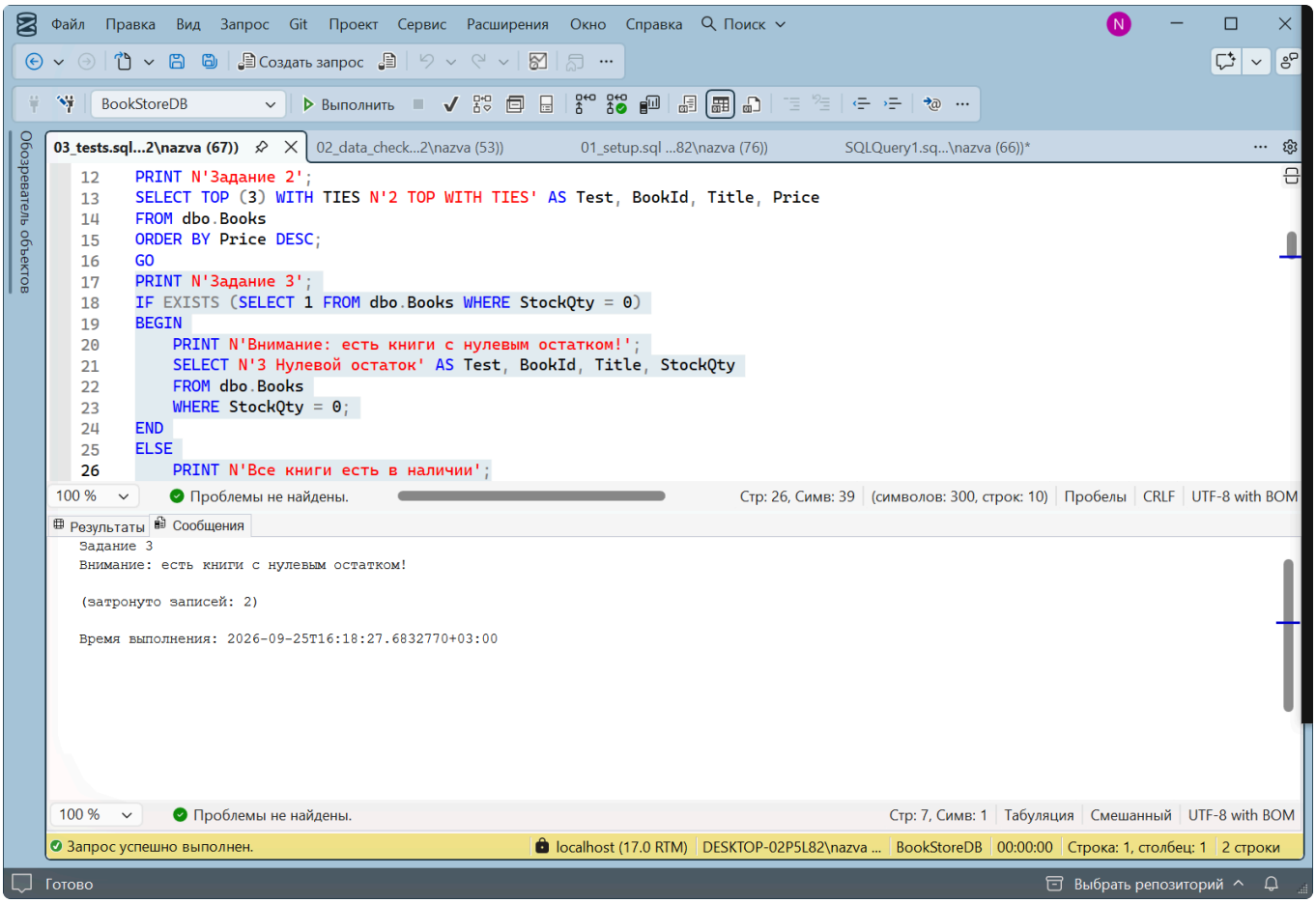
```
PRINT N'Задание 3';
IF EXISTS (SELECT 1 FROM dbo.Books WHERE StockQty = 0)
BEGIN
    PRINT N'Внимание: есть книги с нулевым остатком!';
    SELECT N'3 Нулевой остаток' AS Test, BookId, Title, StockQty
    FROM dbo.Books
    WHERE StockQty = 0;
END
ELSE
    PRINT N'Все книги есть в наличии';
```

Результат: выведено сообщение «Внимание: есть книги с нулевым остатком!» и 2 книги: «Преступление и наказание», «Оно».

Скриншот 5. Список книг с нулевым остатком



Скриншот 6. Сообщение во вкладке Messages



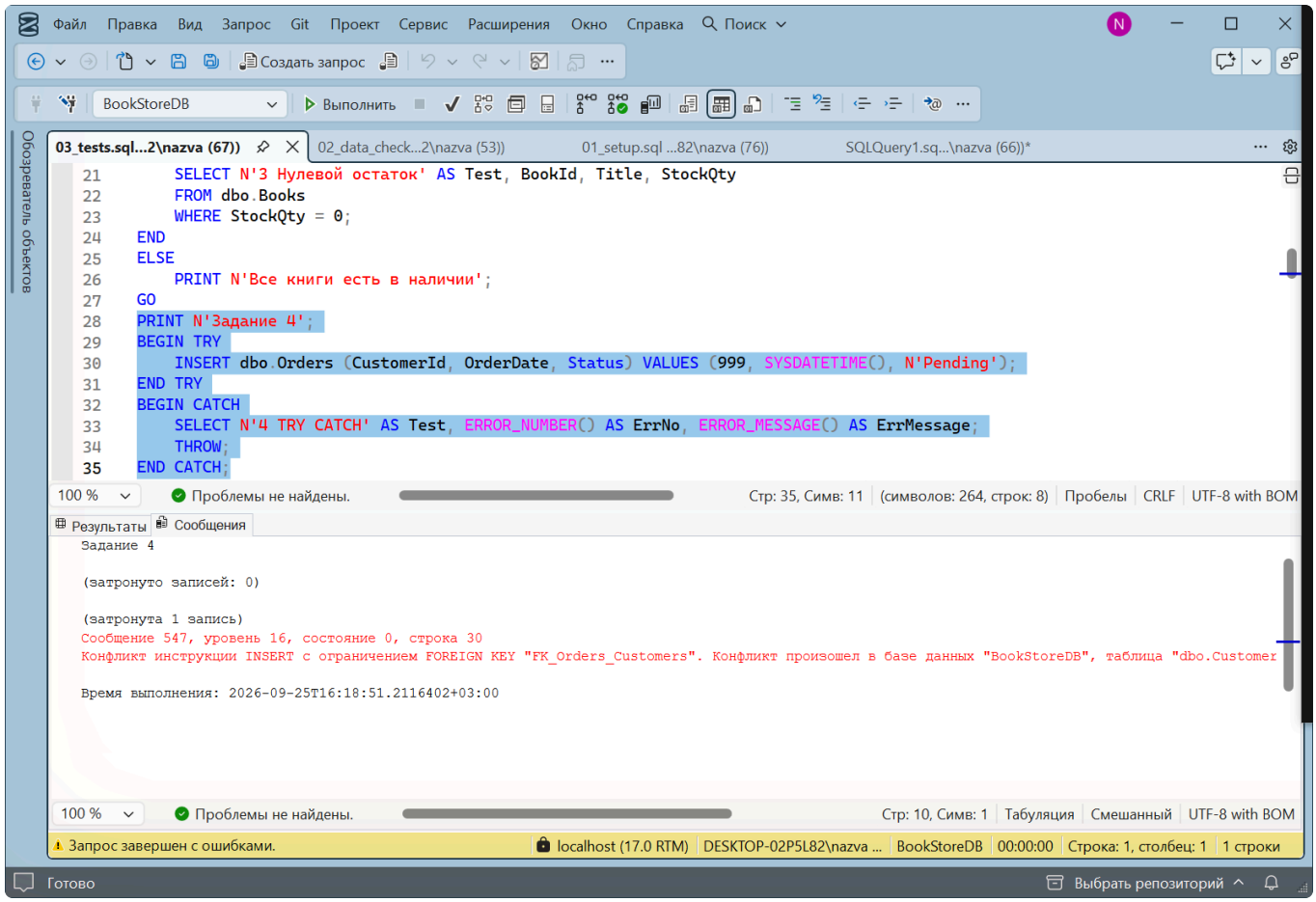
Задание 4. TRY...CATCH + THROW

Код

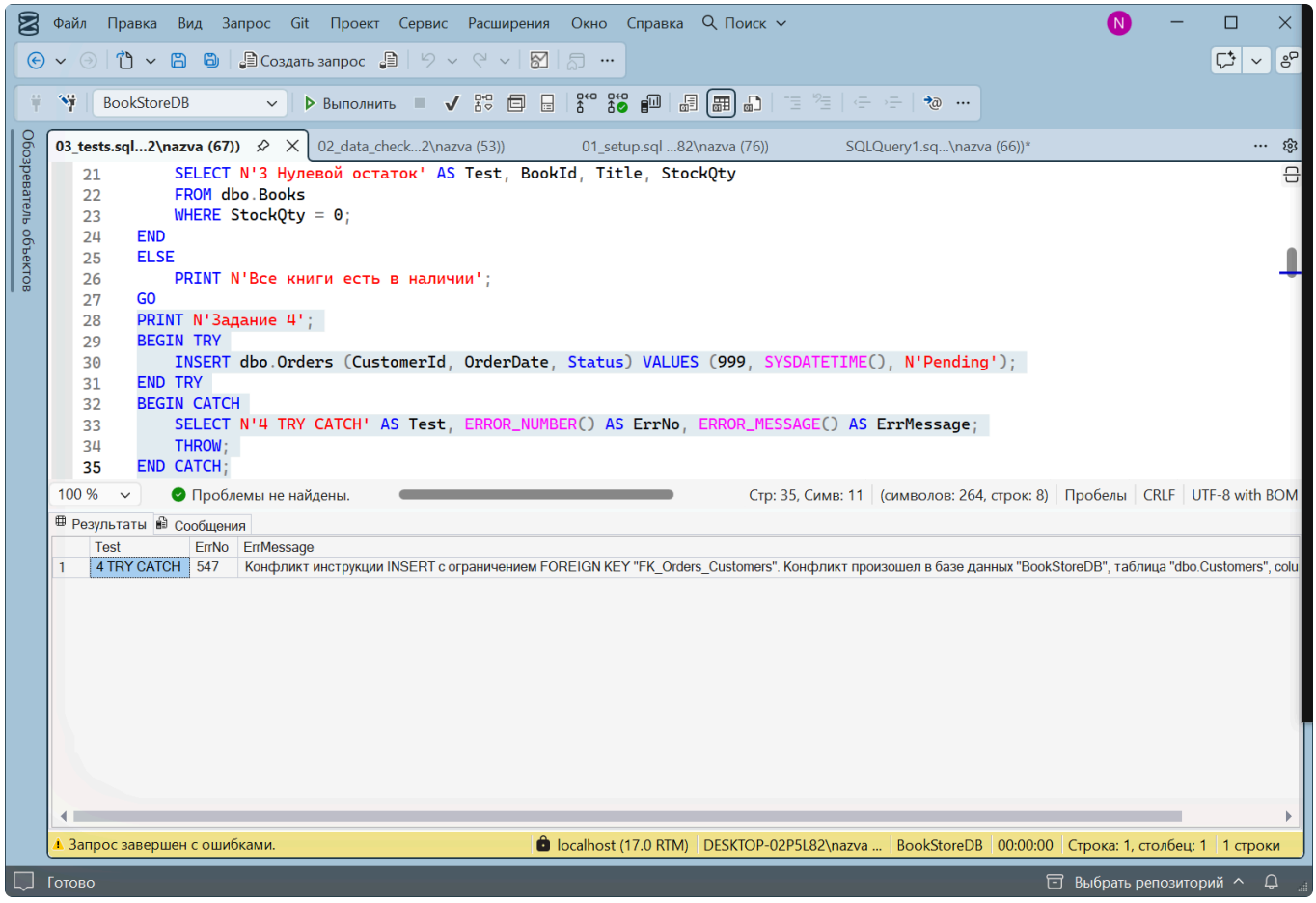
```
PRINT N'Задание 4';
BEGIN TRY
    INSERT dbo.Orders (CustomerId, OrderDate, Status) VALUES (999,
SYSDATETIME(), N'Pending');
END TRY
BEGIN CATCH
    SELECT N'4 TRY CATCH' AS Test, ERROR_NUMBER() AS ErrNo, ERROR_MESSAGE() AS
ErrMessage;
    THROW;
END CATCH;
```

Результат: в CATCH выведены ErrNo = 547 и текст о конфликте с ограничением FOREIGN KEY "FK_Orders_Customers", затем THROW повторно выбросил ту же ошибку 547 (видна красным во вкладке Messages), то есть ошибка не проглочена.

Скриншот 7. Повторно выброшенная ошибка 547 во вкладке Messages



Скриншот 8. Номер и текст ошибки, выведенные в CATCH



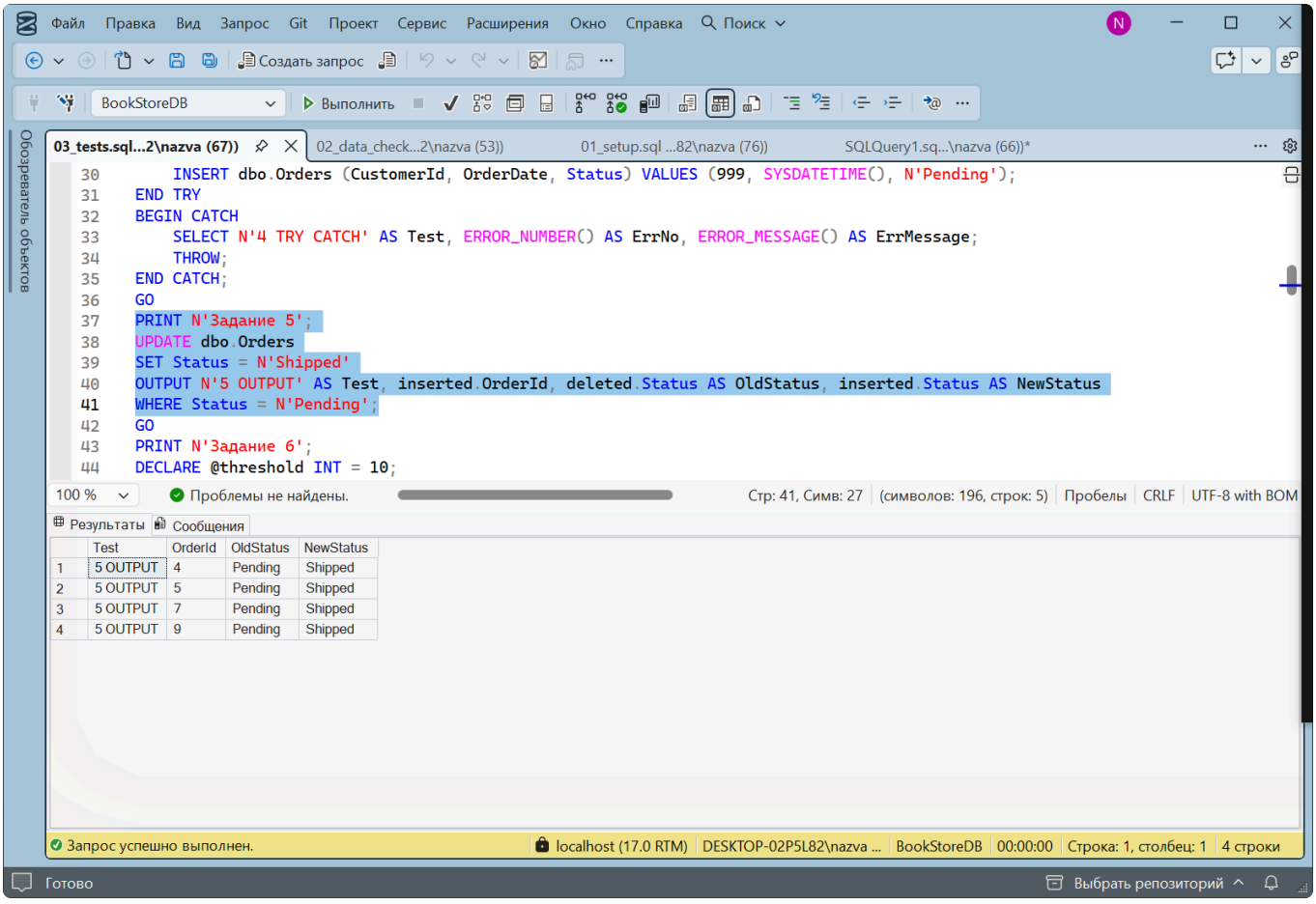
Задание 5. OUTPUT при UPDATE

Код

```
PRINT N'Задание 5';
UPDATE dbo.Orders
SET Status = N'Shipped'
OUTPUT N'5 OUTPUT' AS Test, inserted.OrderId, deleted.Status AS OldStatus,
inserted.Status AS NewStatus
WHERE Status = N'Pending';
```

Результат: обновлено 4 заказа (OrderId 4, 5, 7, 9): OldStatus = Pending, NewStatus = Shipped.

Скриншот 9. OUTPUT со старым и новым статусом



Задание 6. Переменные и WHILE

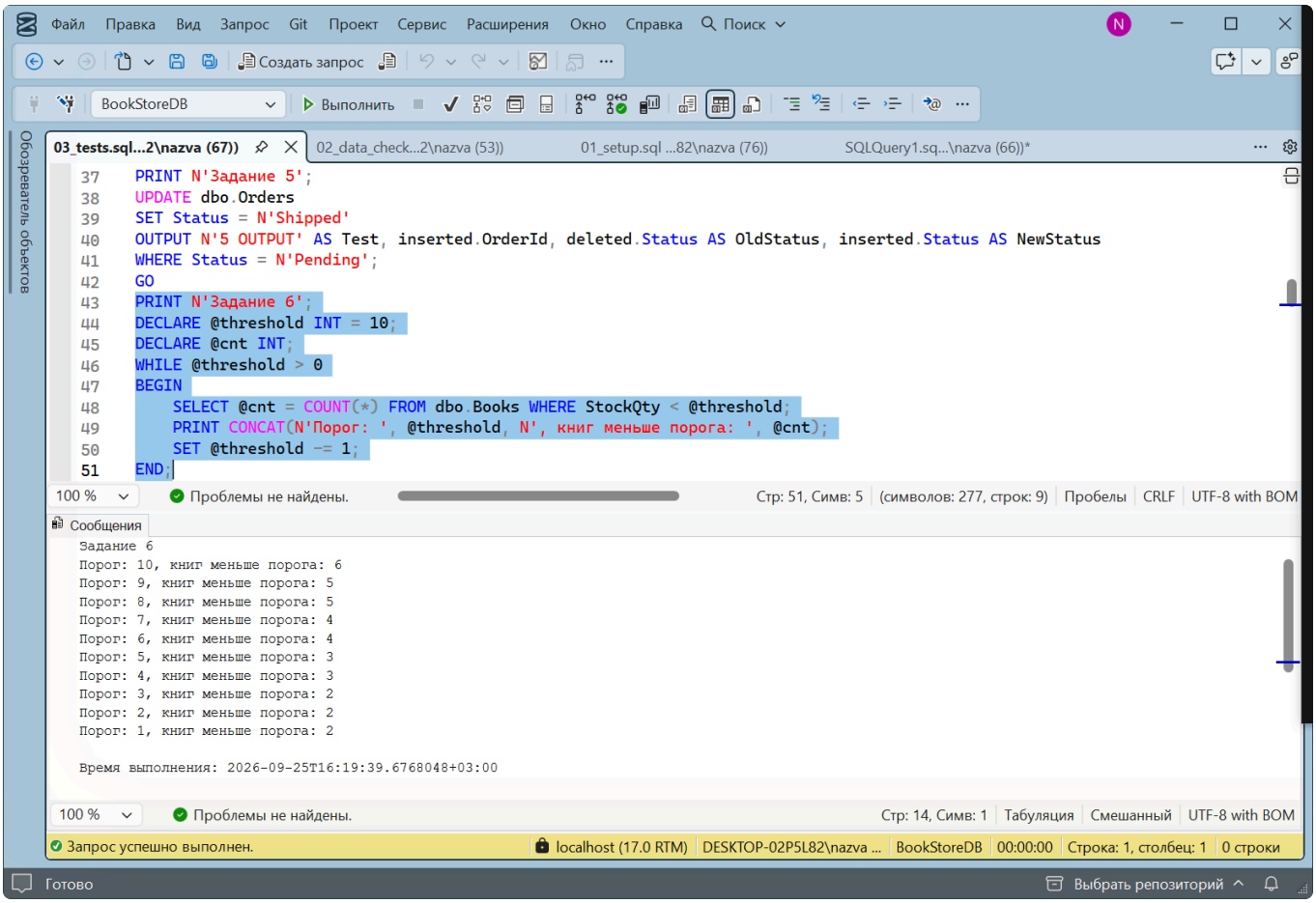
Код

```
PRINT N'Задание 6';
DECLARE @threshold INT = 10;
```

```
DECLARE @cnt INT;
WHILE @threshold > 0
BEGIN
    SELECT @cnt = COUNT(*) FROM dbo.Books WHERE StockQty < @threshold;
    PRINT CONCAT(N'Порог: ', @threshold, N', книг меньше порога: ', @cnt);
    SET @threshold -= 1;
END;
```

Результат: 10 строк от «Порог: 10, книг меньше порога: 6» до «Порог: 1, книг меньше порога: 2», цикл остановился на 0.

Скриншот 10. Вывод цикла WHILE



Задание 7. MERGE

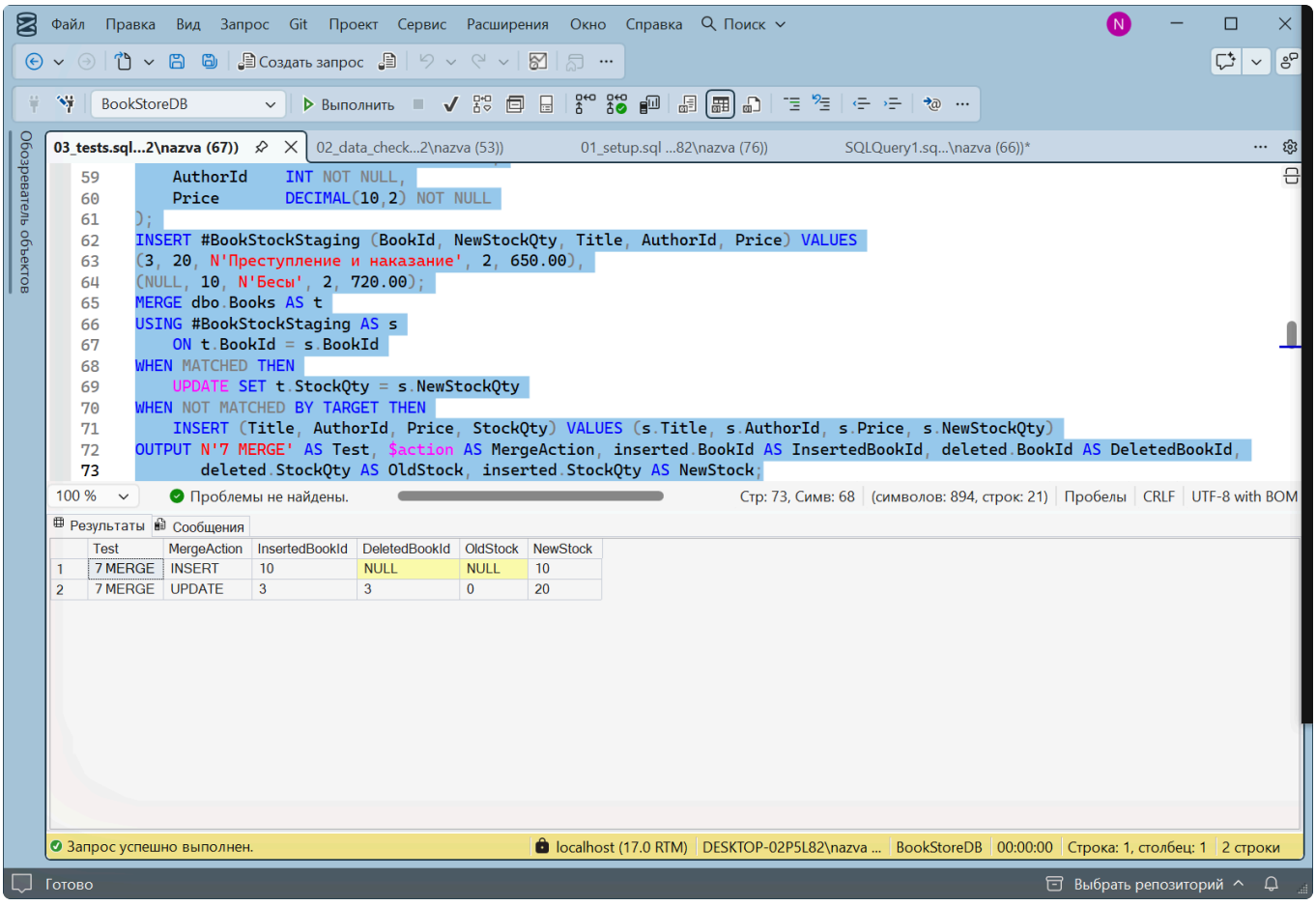
Код

```
PRINT N'Задание 7';
DROP TABLE IF EXISTS #BookStockStaging;
CREATE TABLE #BookStockStaging (
    BookId      INT NULL,
    NewStockQty INT NOT NULL,
    Title       NVARCHAR(200) NOT NULL,
```

```
    AuthorId    INT NOT NULL,
    Price       DECIMAL(10,2) NOT NULL
);
INSERT #BookStockStaging (BookId, NewStockQty, Title, AuthorId, Price) VALUES
(3, 20, N'Преступление и наказание', 2, 650.00),
(NULL, 10, N'Бесы', 2, 720.00);
MERGE dbo.Books AS t
USING #BookStockStaging AS s
    ON t.BookId = s.BookId
WHEN MATCHED THEN
    UPDATE SET t.StockQty = s.NewStockQty
WHEN NOT MATCHED BY TARGET THEN
    INSERT (Title, AuthorId, Price, StockQty) VALUES (s.Title, s.AuthorId,
s.Price, s.NewStockQty)
OUTPUT N'7 MERGE' AS Test, $action AS MergeAction, inserted.BookId AS
InsertedBookId, deleted.BookId AS DeletedBookId,
       deleted.StockQty AS OldStock, inserted.StockQty AS NewStock;
```

Результат: 2 действия: UPDATE для BookId 3 (остаток 0 -> 20) и INSERT новой книги «Бесы» с BookId 10 (deleted.BookId = NULL).

Скриншот 11. Результат MERGE с $action



Задание 8. EXISTS vs IN

Код (EXISTS)

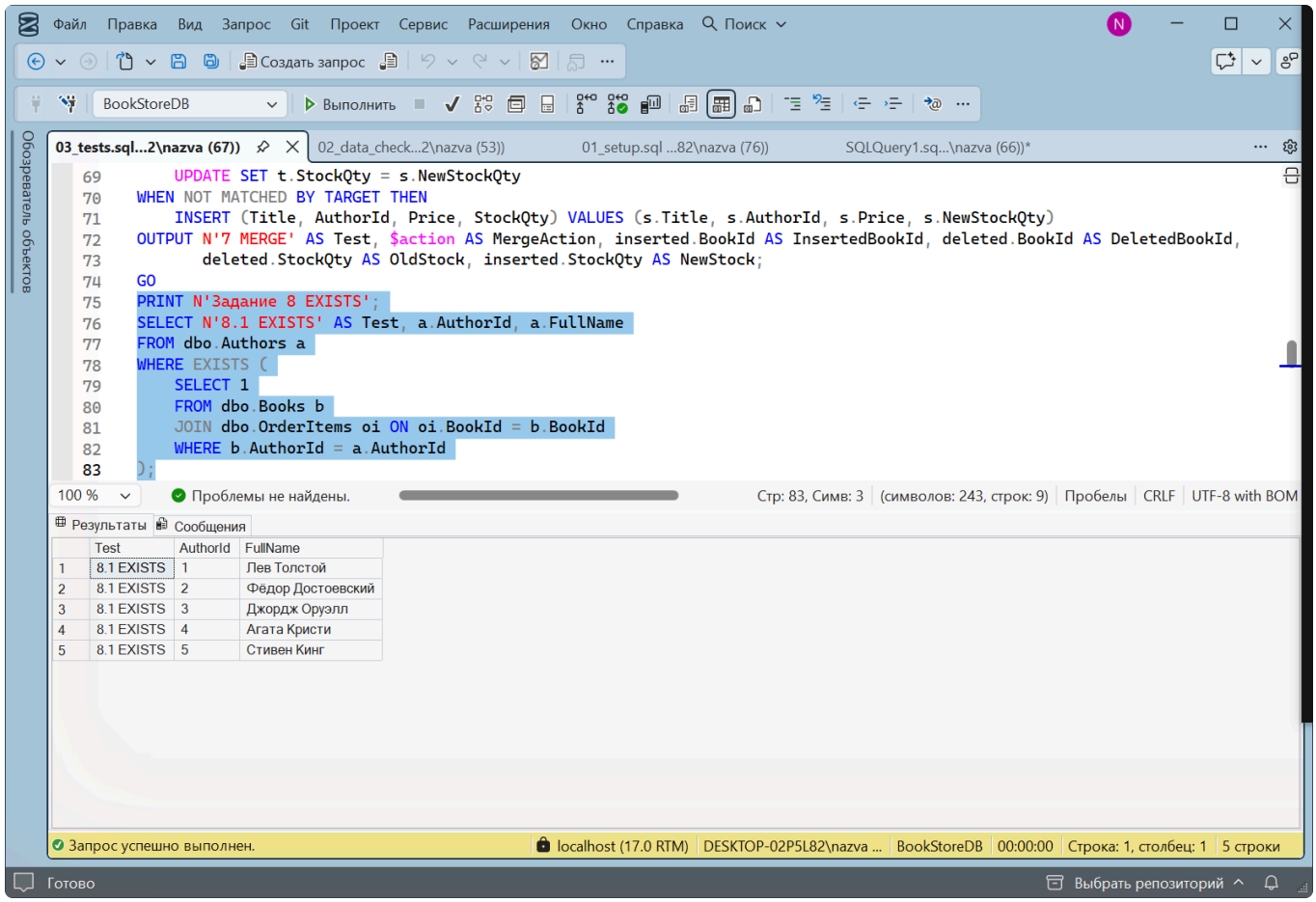
```
PRINT N'Задание 8 EXISTS';
SELECT N'8.1 EXISTS' AS Test, a.AuthorId, a.FullName
FROM dbo.Authors a
WHERE EXISTS (
    SELECT 1
    FROM dbo.Books b
    JOIN dbo.OrderItems oi ON oi.BookId = b.BookId
    WHERE b.AuthorId = a.AuthorId
);
```

Код (IN)

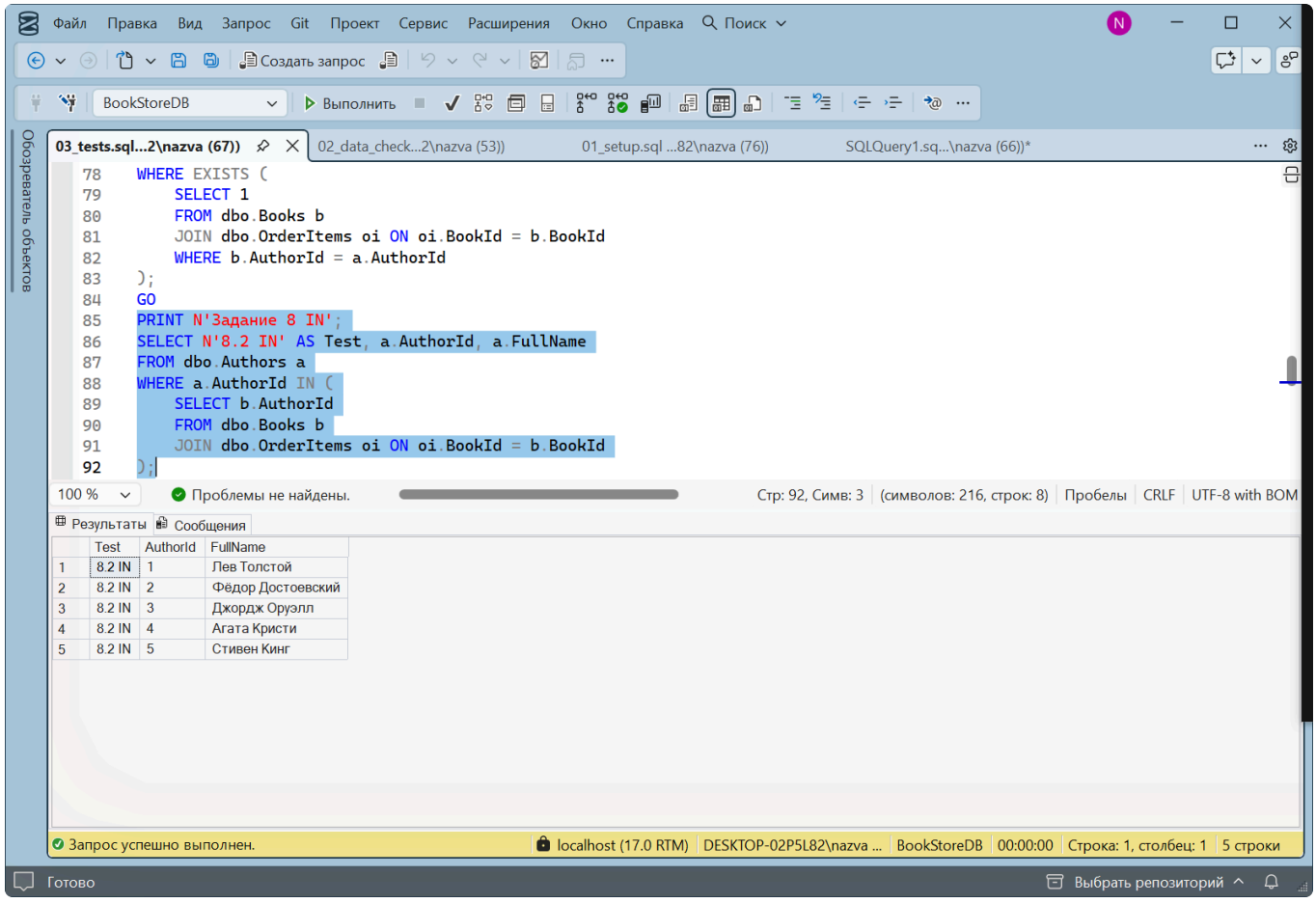
```
PRINT N'Задание 8 IN';
SELECT N'8.2 IN' AS Test, a.AuthorId, a.FullName
FROM dbo.Authors a
WHERE a.AuthorId IN (
    SELECT b.AuthorId
    FROM dbo.Books b
    JOIN dbo.OrderItems oi ON oi.BookId = b.BookId
);
```

Результат: оба запроса вернули одинаковых 5 авторов (Толстой, Достоевский, Оруэлл, Кристи, Кинг). Булгаков не попал - его книгу не заказывали.

Скриншот 12. Результат через EXISTS

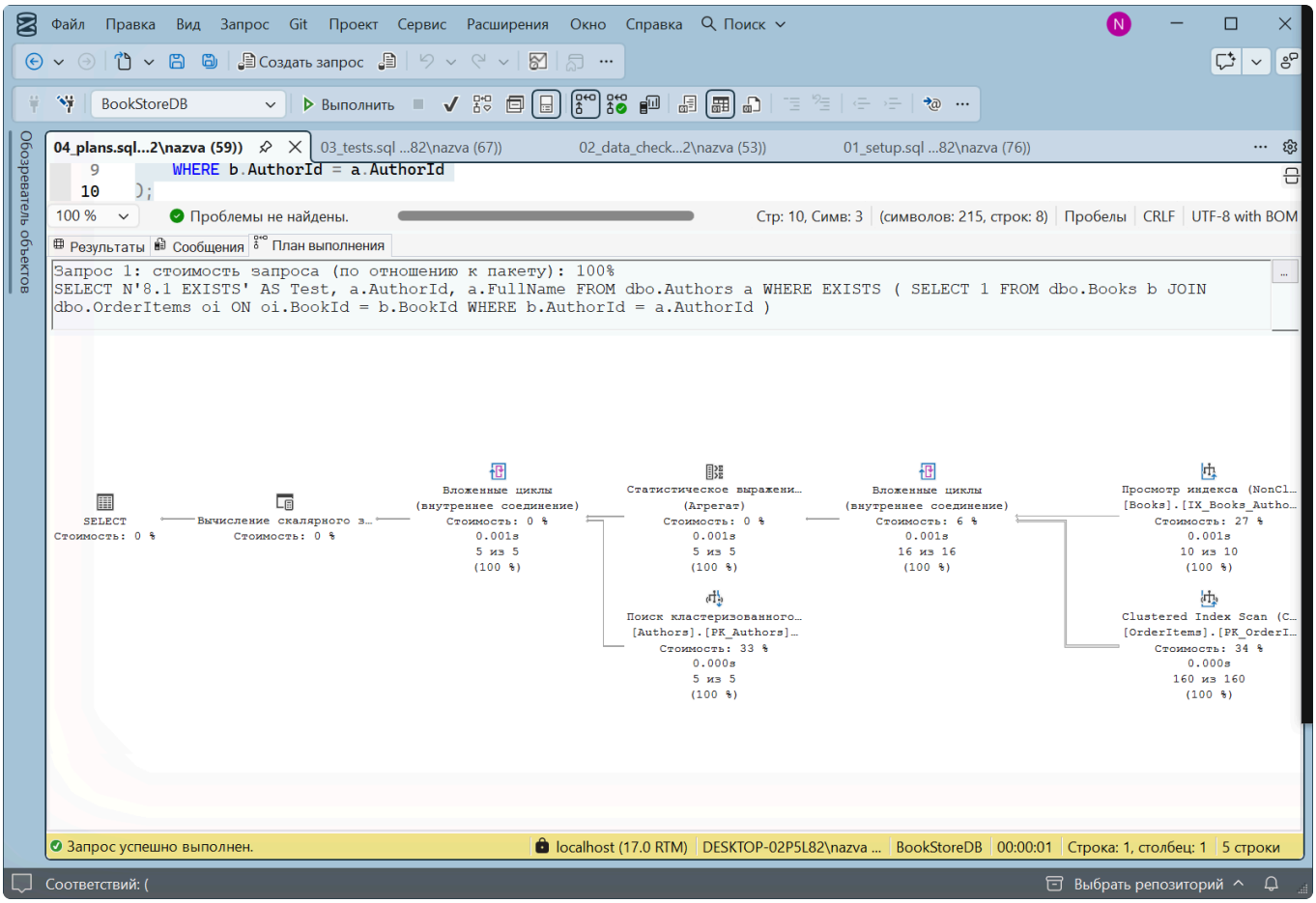


Скриншот 13. Результат через IN

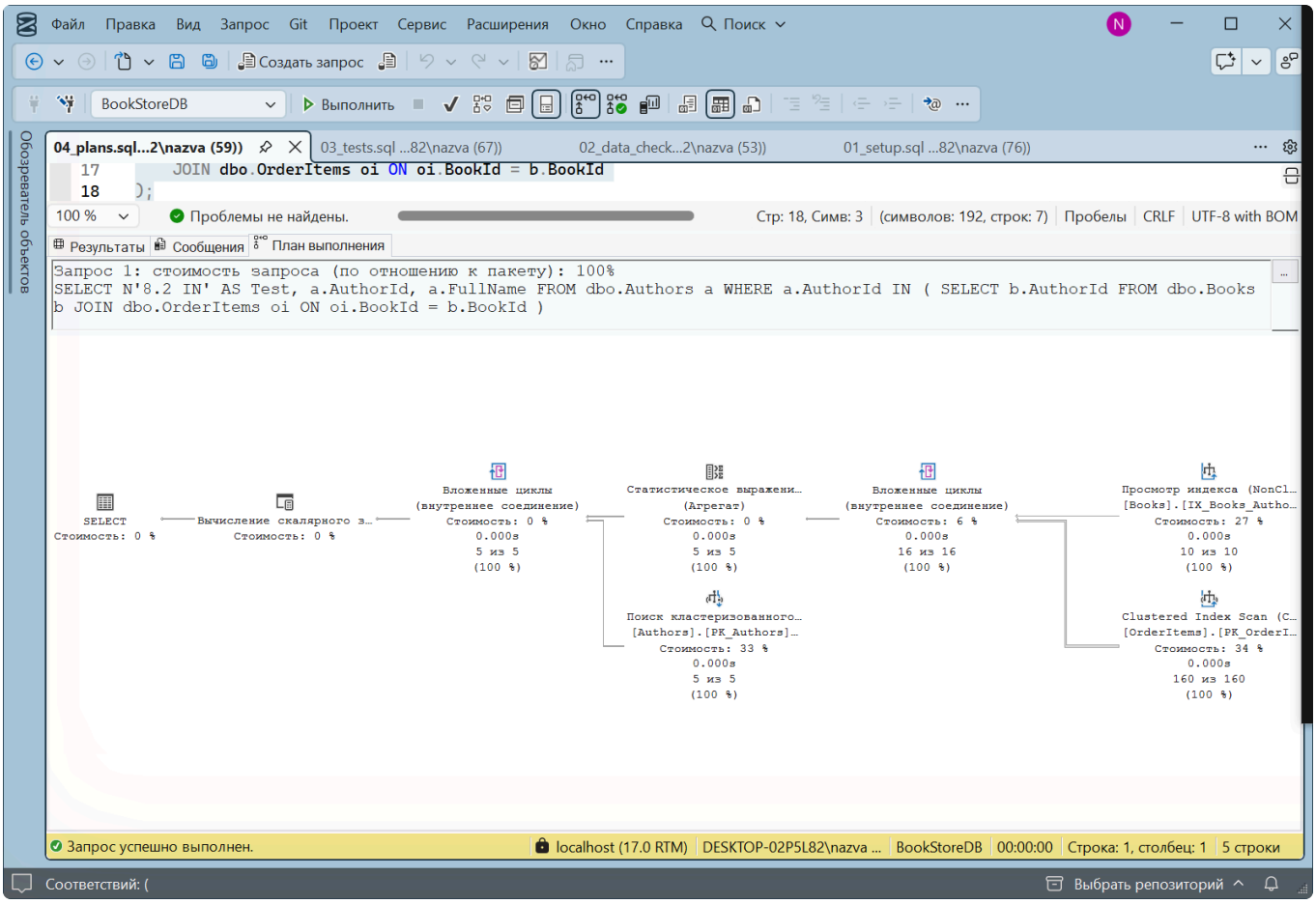


Планы выполнения (Ctrl+M)

Скриншот 17. План запроса с EXISTS



Скриншот 18. План запроса с IN



Планы одинаковые: оптимизатор переписал IN в полусоединение, так же как EXISTS.

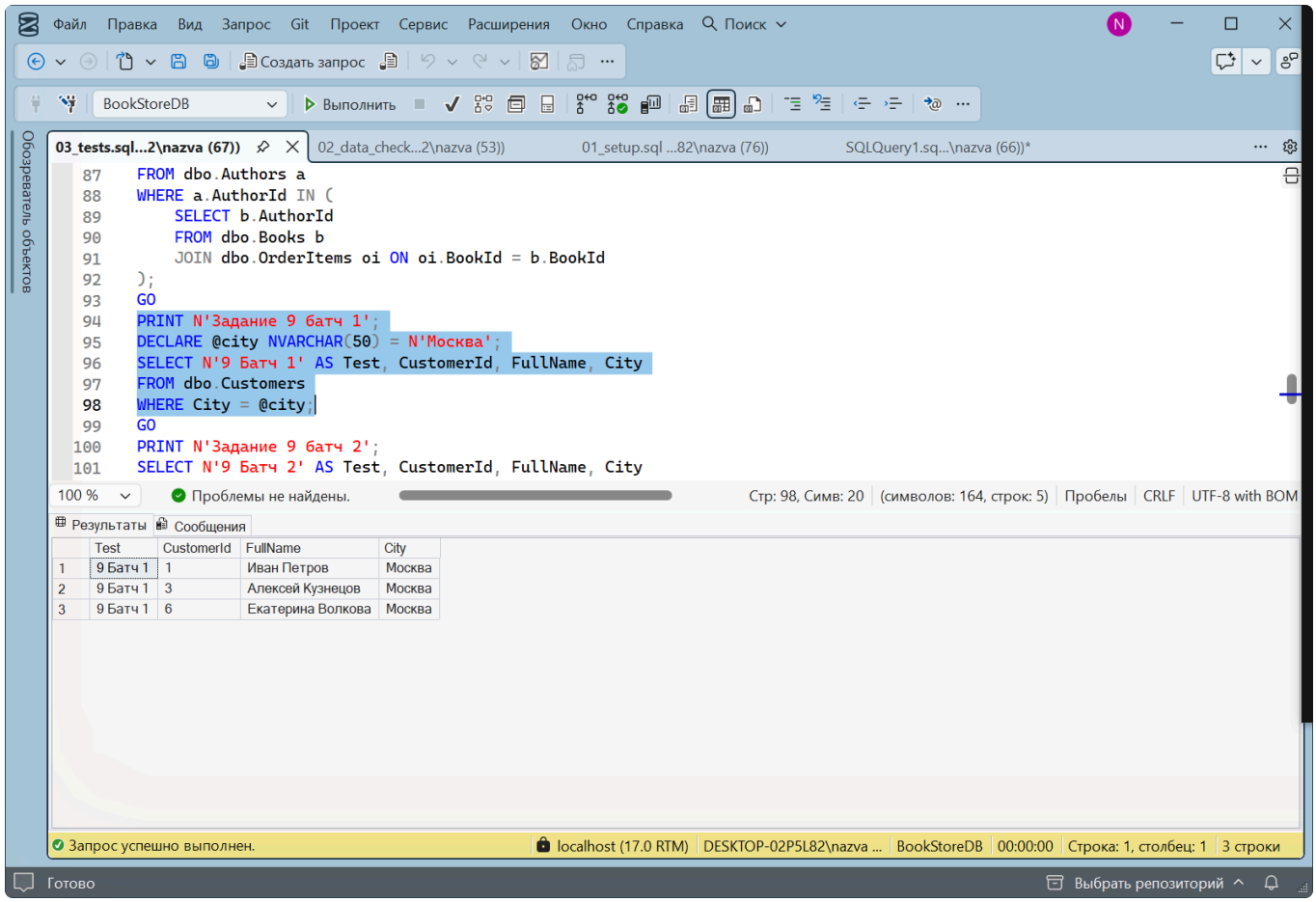
Задание 9. GO и область видимости

Код

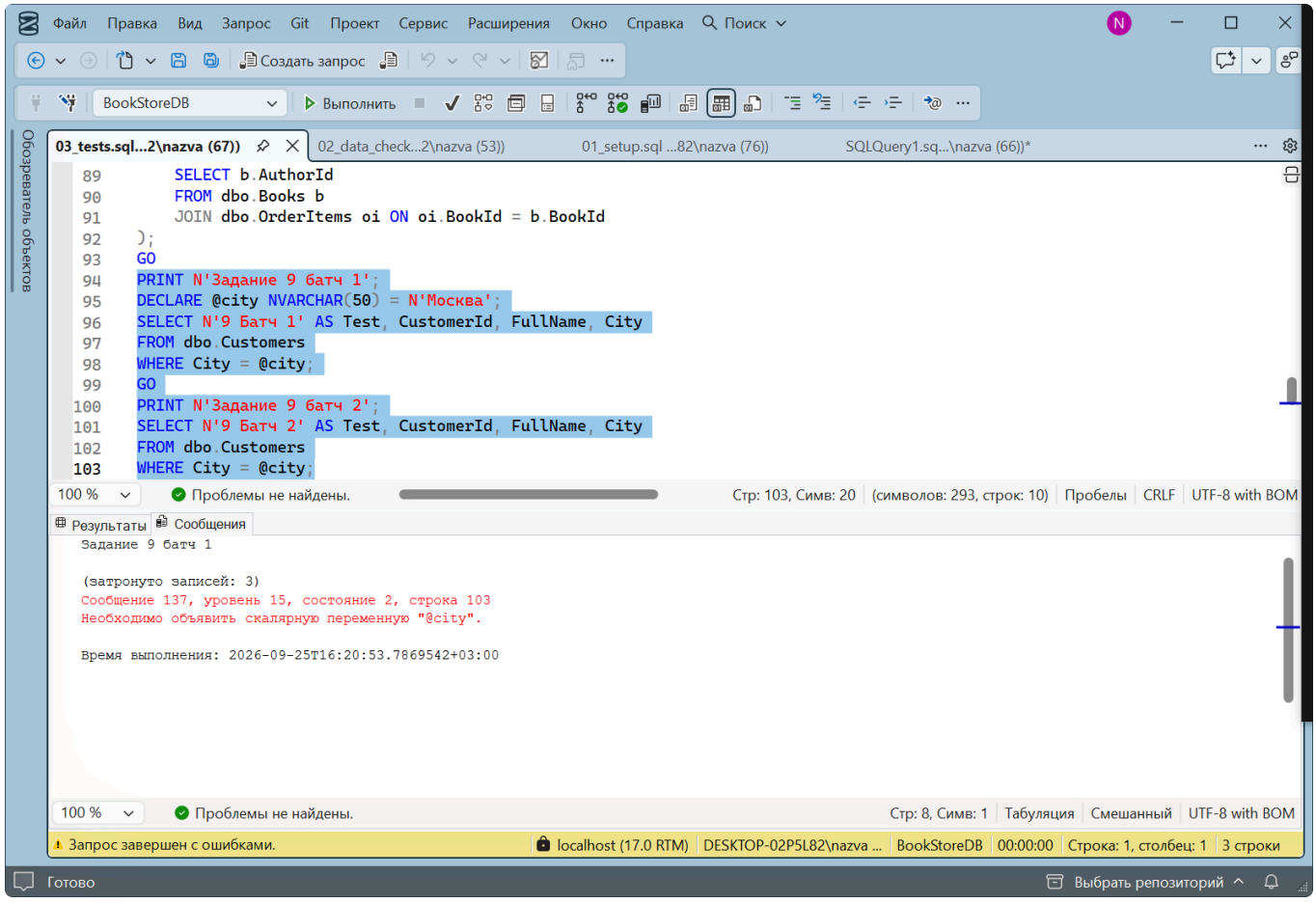
```
PRINT N'Задание 9 батч 1';
DECLARE @city NVARCHAR(50) = N'Москва';
SELECT N'9 Батч 1' AS Test, CustomerId, FullName, City
FROM dbo.Customers
WHERE City = @city;
GO
PRINT N'Задание 9 батч 2';
SELECT N'9 Батч 2' AS Test, CustomerId, FullName, City
FROM dbo.Customers
WHERE City = @city;
```

Результат: первый батч вывел 3 клиентов из Москвы, второй батч упал с ошибкой 137 «Необходимо объявить скалярную переменную "@city"».

Скриншот 14. Первый батч - клиенты из Москвы



Скриншот 15. Оба батча: во втором ошибка 137



Во втором батче переменная @city не существует, поэтому сервер не смог скомпилировать запрос и выдал ошибку 137. GO не является командой T-SQL, это разделитель пакетов для SSMS, и каждый пакет отправляется на сервер отдельно. Локальная переменная живёт только внутри своего пакета, поэтому после GO её уже нет.

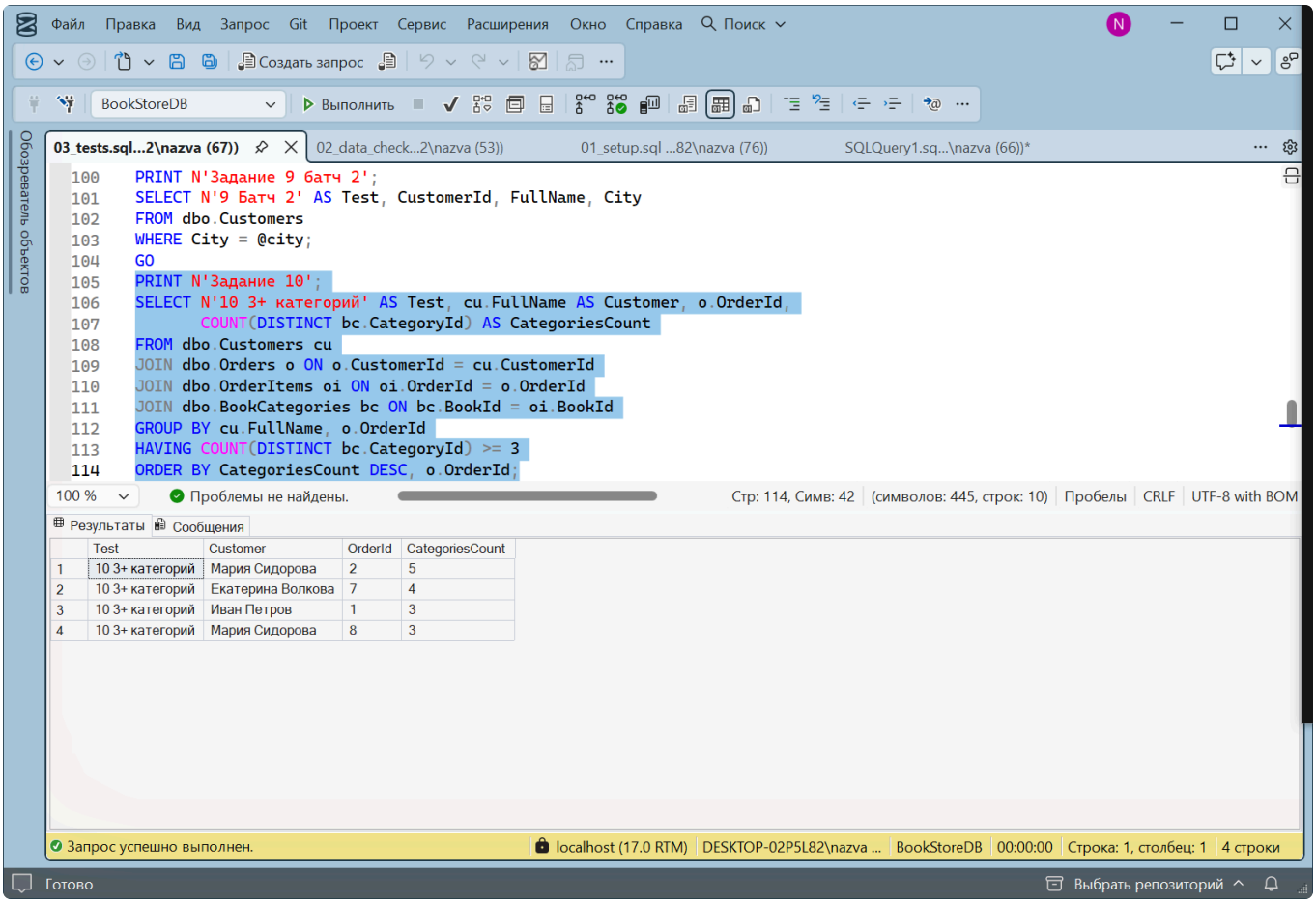
Задание 10. Клиенты, купившие книги из 3+ категорий в одном заказе

Код

```
PRINT N'Задание 10';
SELECT N'10 3+ категорий' AS Test, cu.FullName AS Customer, o.OrderId,
       COUNT(DISTINCT bc.CategoryId) AS CategoriesCount
FROM dbo.Customers cu
JOIN dbo.Orders o ON o.CustomerId = cu.CustomerId
JOIN dbo.OrderItems oi ON oi.OrderId = o.OrderId
JOIN dbo.BookCategories bc ON bc.BookId = oi.BookId
GROUP BY cu.FullName, o.OrderId
HAVING COUNT(DISTINCT bc.CategoryId) >= 3
ORDER BY CategoriesCount DESC, o.OrderId;
```

Результат: 4 строки: Мария Сидорова - заказ 2 (5 категорий), Екатерина Волкова - заказ 7 (4), Иван Петров - заказ 1 (3), Мария Сидорова - заказ 8 (3).

Скриншот 16. Клиенты и заказы с 3 и более категориями



Запрос работает потому, что схема нормализована: каждая книга заказа лежит отдельной строкой в OrderItems, а каждая категория книги - отдельной строкой в BookCategories. Благодаря этому можно соединить заказ с категориями через JOIN и посчитать COUNT(DISTINCT CategoryId) по заказу. Если бы категории хранились строкой через запятую, а книги заказа в одном поле, посчитать уникальные категории обычным JOIN было бы нельзя.

Задание 11. Index Seek / Key Lookup

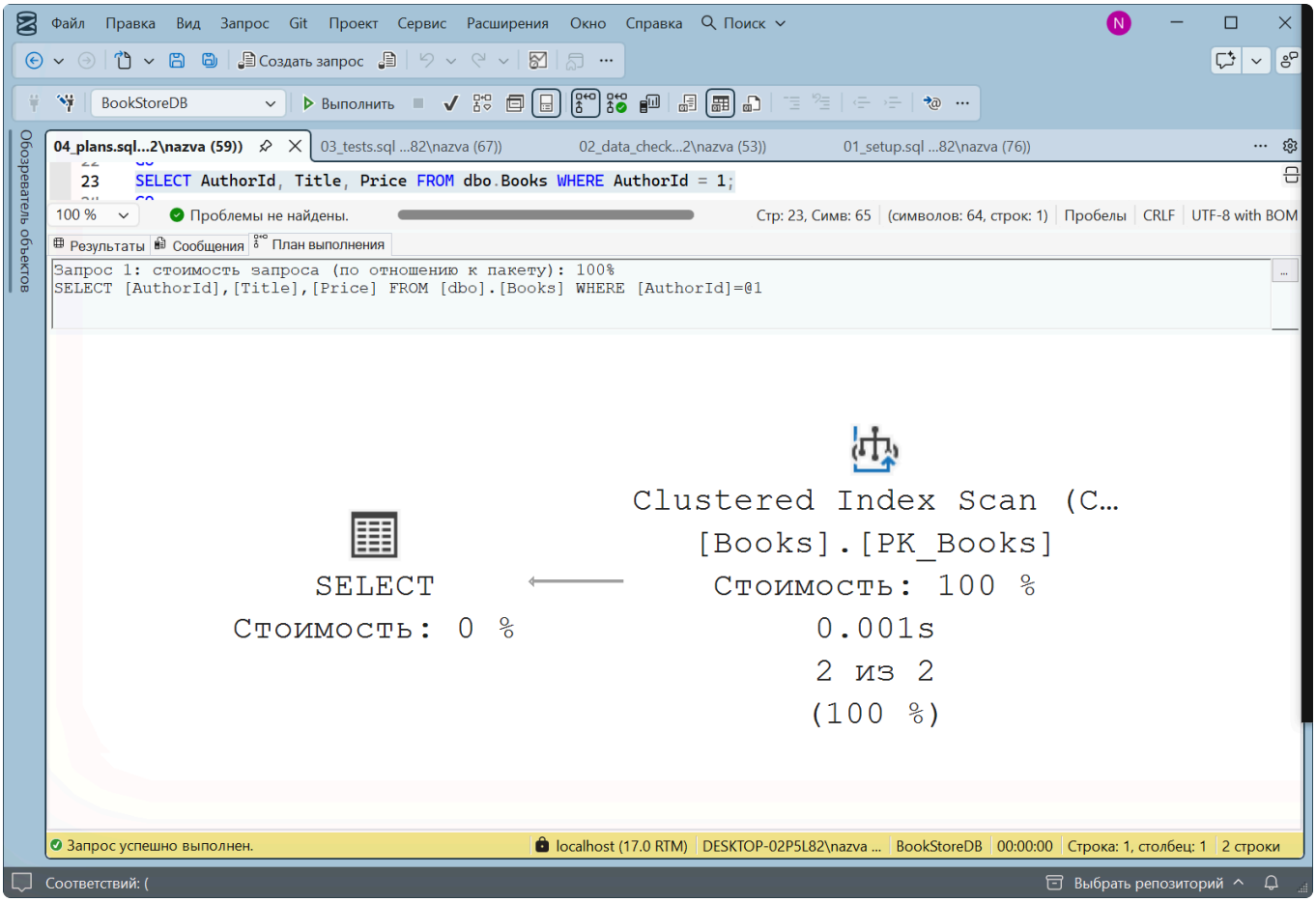
Код

```
DROP INDEX IF EXISTS IX_Books_AuthorId_Covering ON dbo.Books;
DROP INDEX IF EXISTS IX_Books_AuthorId ON dbo.Books;
GO
SELECT AuthorId, Title, Price FROM dbo.Books WHERE AuthorId = 1;
GO
CREATE NONCLUSTERED INDEX IX_Books_AuthorId ON dbo.Books (AuthorId);
GO
SELECT AuthorId, Title, Price FROM dbo.Books WITH (INDEX(IX_Books_AuthorId))
WHERE AuthorId = 1;
GO
DROP INDEX IX_Books_AuthorId ON dbo.Books;
```

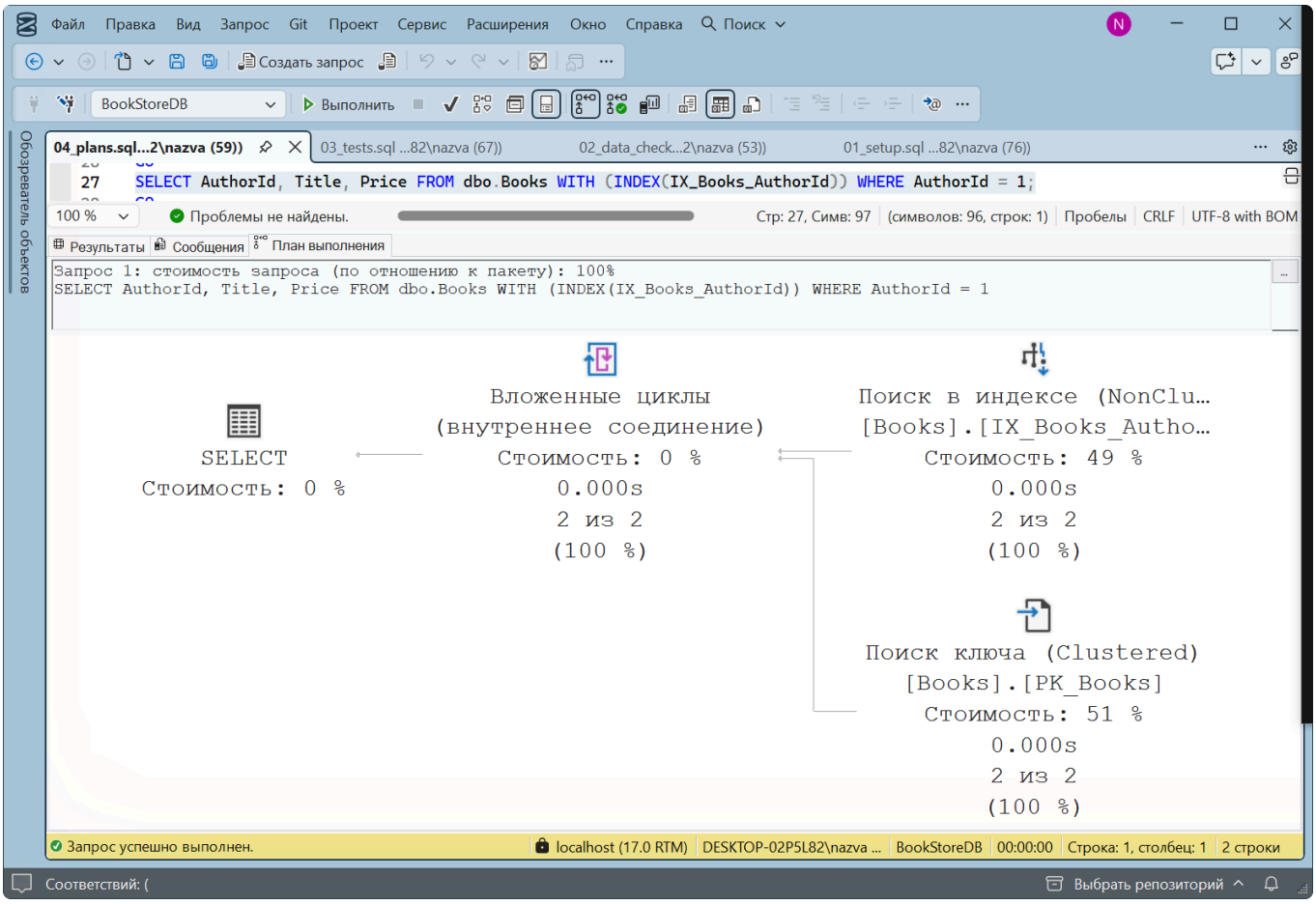
```
CREATE NONCLUSTERED INDEX IX_Books_AuthorId_Covering ON dbo.Books (AuthorId)
INCLUDE (Title, Price, StockQty);
GO
SELECT AuthorId, Title, Price FROM dbo.Books WHERE AuthorId = 1;
```

- **До покрывающего индекса:** индекса по AuthorId нет, поэтому будет Clustered Index Scan (при обычном некластерном индексе по AuthorId без INCLUDE - Index Seek + Key Lookup за Title и Price).
- **После покрывающего индекса:** только Index Seek по IX_Books_AuthorId_Covering, без Key Lookup, так как все нужные столбцы есть в индексе.

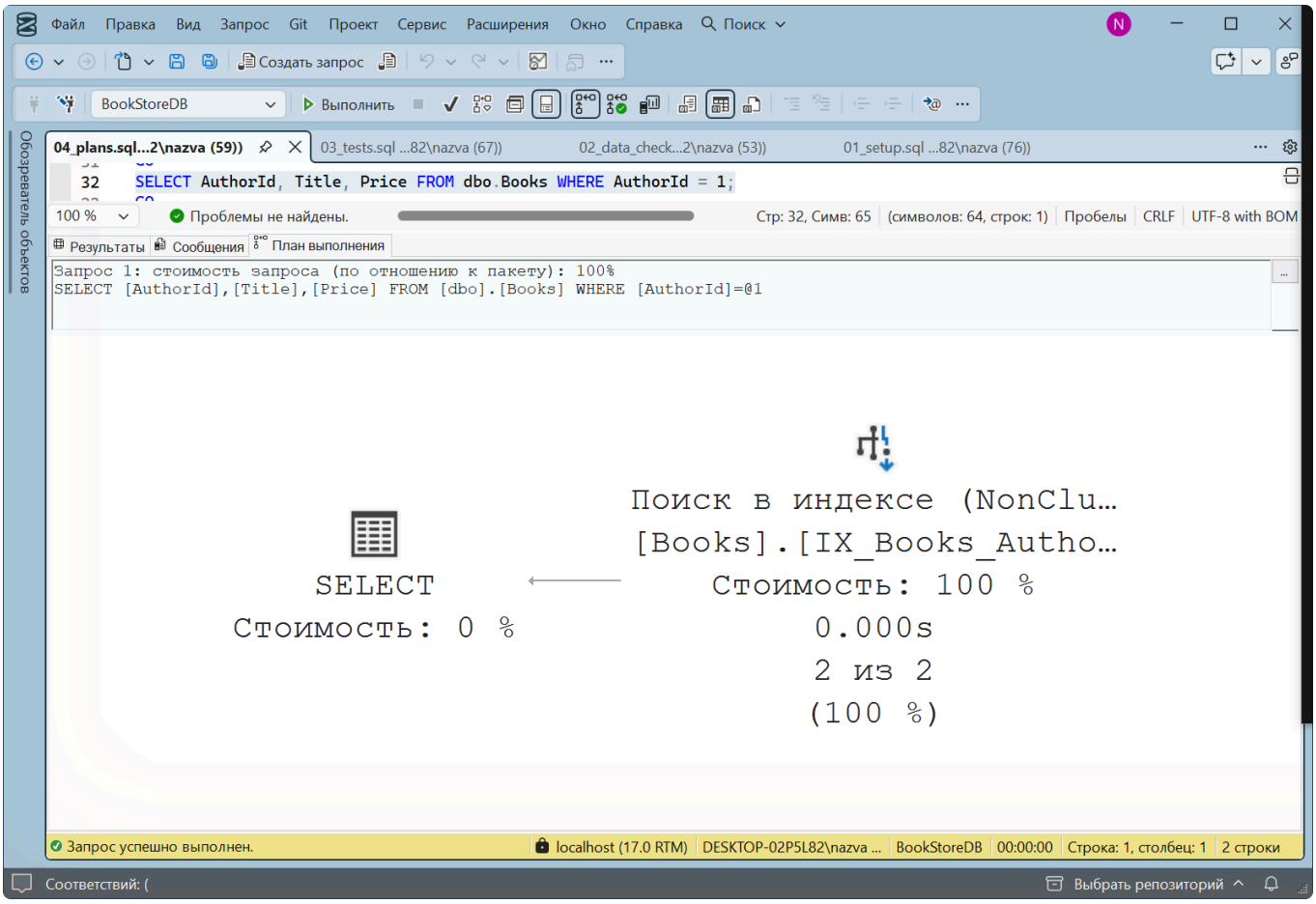
Скриншот 19. До: индекса по AuthorId нет - Clustered Index Scan



Скриншот 20. Обычный индекс по AuthorId (с подсказкой INDEX, так как таблица маленькая) - Index Seek + Key Lookup



Скриншот 21. После: покрывающий индекс - только Index Seek


Файл Правка Вид Запрос Git Проект Сервис Расширения Окно Справка Поиск
Создать запрос
BookStoreDB
Выполнить
04_plans.sql...2\nazva (59))
03_tests.sql ...82\nazva (67))
02_data_check...2\nazva (53))
01_setup.sql ...82\nazva (76))
32 SELECT AuthorId, Title, Price FROM dbo.Books WHERE AuthorId = 1;
100 %
Проблемы не найдены.
Стр: 32, Симв: 65
(символов: 64, строк: 1)
Пробелы
CRLF
UTF-8 with BOM
Обозреватель объектов
Результаты
Сообщения
План выполнения
Запрос 1: стоимость запроса (по отношению к пакету): 100%
SELECT [AuthorId],[Title],[Price] FROM [dbo].[Books] WHERE [AuthorId]=@1
SELECT
Стоимость: 0 %
Поиск в индексе (NonClu…
[Books].[IX_Books_Autho…
Стоимость: 100 %
0.000s
2 из 2
(100 %)
Запрос успешно выполнен.
localhost (17.0 RTM)
DESKTOP-02P5L82\nazva ...
BookStoreDB
00:00:00
Строка: 1, столбец: 1
2 строки
Соответствий: (
Выбрать репозиторий